**Task 1: Guided Activity, Library item model**

```
import Foundation
enum BookStatus {
    case available
    case borrowed(by: String, dueIns: Int)
    case reserved(for: String)
    var label: String {
        switch self {
        case .available:
            return "Available"
        case .borrowed(let name, let days):
            return "Borrowed by \(name), due in \(days) days"
        case .reserved(let name):
            return "Reserved for \(name)"
        }
    }
}
struct Book {
    var title: String
    var status: BookStatus = .available
}

class Member {
    var name: String
    var borrowedBooks: [String]

    init(name: String, borrowedBooks: [String] = []) {
        self.name = name
        self.borrowedBooks = borrowedBooks
    }
}

var book1 = Book(title: "Swift Basics")
var book2 = book1
book2.status = .borrowed(by: "Anu", dueIns: 5)
print("Struct Copy:")
print("\(book1.title): \(book1.status.label)")
print("\(book2.title): \(book2.status.label)")

let member1 = Member(name: "Ben", borrowedBooks: ["Swift Basics"])
let member2 = member1
member2.name = "Chitra"
print("\nClass Reference:")
print("Member 1 name: \(member1.name)")
print("Member 2 name: \(member2.name)")
```

**OUTPUT:**

```
Struct Copy:
Swift Basics: Available
Swift Basics: Borrowed by Anu, due in 5 days

Class Reference:
Member 1 name: Chitra
Member 2 name: Chitra
```

**Task 2: Practice Task, Shapes and areas**

```
import Foundation
enum Shape {
    case circle(radius: Double)
    case rectangle(width: Double, height: Double)
    case triangle(base: Double, height: Double)
    var area: Double {
        switch self {
        case .circle(let radius):
            return Double.pi * radius * radius

        case .rectangle(let width, let height):
            return width * height

        case .triangle(let base, let height):
            return 0.5 * base * height
        }
    }
}

let circle = Shape.circle(radius: 5)
let rectangleShape = Shape.rectangle(width: 4, height: 6)
let triangle = Shape.triangle(base: 8, height: 5)
print("Circle Area:", circle.area)
print("Rectangle Area:", rectangleShape.area)
print("Triangle Area:", triangle.area)

struct Rectangle {
    var width: Double
    var height: Double
    var perimeter: Double {
        return 2 * (width + height)
    }
    mutating func scale(by factor: Double) {
        width *= factor
        height *= factor
    }
}
```

```
var original = Rectangle(width: 10, height: 5)
var copy = original
print("\nBefore Scaling:")
print("Original Perimeter:", original.perimeter)
print("Copy Perimeter:", copy.perimeter)
copy.scale(by: 2)
print("\nAfter Scaling Copy:")
print("Original Width:", original.width)
print("Original Height:", original.height)
print("Original Perimeter:", original.perimeter)
print("Copy Width:", copy.width)
print("Copy Height:", copy.height)
print("Copy Perimeter:", copy.perimeter)
```

**OUTPUT:**

```
Circle Area: 78.53981633974483
Rectangle Area: 24.0
Triangle Area: 20.0

Before Scaling:
Original Perimeter: 30.0
Copy Perimeter: 30.0

After Scaling Copy:
Original Width: 10.0
Original Height: 5.0
Original Perimeter: 30.0
Copy Width: 20.0
Copy Height: 10.0
Copy Perimeter: 60.0
```

**Task 3: Challenge Task, Why does this not compile?**

**Correct Swift Playground Code**

The compile error occurs at c.increment() because c is declared using let, making the struct instance immutable.

```
import Foundation
struct Counter {
   var value = 0
   mutating func increment() {
      value += 1
   }}
class Box {
   var value = 0
   func increment() {
      value += 1
   }}
var c = Counter()
```

c.increment()
print("Counter value:", c.value)
let b = Box()
b.increment()
print("Box value:", b.value)
let b1 = Box()
let b2 = b1
print("Same object:", b1 === b2)

**Reason:** Counter is a struct, which is a value type. Its increment() method is marked mutating because it changes the stored property value. A mutating method cannot be called on a struct instance declared with let.

```
var c = Counter()
c.increment()
```

Using `var` allows the struct instance to be modified.

**Why doesn't the class have the same problem?**

let b = Box()
b.increment()

This works because Box is a class, which is a reference type. The let constant prevents b from referring to a different object, but it does not prevent changes to the object's var properties.

Unlike structs, class methods do not need the mutating keyword.

**Using === to compare class references**

let b1 = Box()
let b2 = b1
print(b1 === b2)

The === operator checks whether two class references point to the exact same object in memory. Since b2 references the object created for b1, the result is true.

**OUTPUT:**

```
Counter value: 1
Box value: 1
Same object: true
```